ebnf2tikz User Manual

Larry D. Pyeatt

March 4, 2026

Contents

1 Introduction **3**

2 Installation **3**
2.1 Prerequisites . . . 3
2.2 Building . . . 3

3 Quick Start **3**

4 Command-Line Options **4**

5 EBNF Syntax Reference **4**
5.1 Productions . . . 4
5.2 Constructs . . . 5
5.3 Comments . . . 5

6 Annotations **5**
6.1 `subsume` . . . 5
6.2 `subsume as` . . . 5
6.3 `caption` . . . 6
6.4 `label` . . . 6
6.5 `sideways` . . . 6

7 LaTeX Integration **6**
7.1 Package Options . . . 7
7.2 Providing EBNF Input . . . 7
7.3 Placing Diagrams . . . 7
7.4 Multi-Pass Workflow . . . 7
7.5 Customizable Lengths . . . 8
7.6 Customizable Styles . . . 8

8 Auto-Wrap **9**

9 Examples **9**
9.1 Simple Productions . . . 9
9.2 Repetition (Loops) . . . 9
9.3 Subsumption . . . 10
9.4 Complex Example with Nested Constructs . . . 11

10 Customization Examples **11**
10.1 Default Appearance 11
10.2 Changing Line Weight 12
10.3 Changing Node Colors 12
10.4 Changing Fonts 12
10.5 Changing Spacing 13
10.6 Replacing the raildiagram Style Entirely 13
10.7 Coexistence with Other TikZ Packages 13

11 Tips **14**

1 Introduction

`ebnf2tikz` is an optimizing compiler that converts Extended Backus-Naur Form (EBNF) grammar specifications into railroad (syntax) diagrams expressed as LaTeX TikZ commands. It supports annotations that control inlining of productions, figure captions, and rotated figures. An optimizer simplifies the resulting abstract syntax tree before layout, producing compact, readable diagrams.

`ebnf2tikz` is free software distributed under the GNU General Public License, version 3 or later.

2 Installation

2.1 Prerequisites

- A C++ compiler with C++17 support (e.g., `g++`)
- Bison $\geq$ 3.7
- Flex
- CMake $\geq$ 3.16
- `pdflatex` with TikZ (for rendering diagrams)

2.2 Building

```
cmake .
make
```

This produces the `ebnf2tikz` executable, `ebnf2tikz.sty` (the LaTeX package), and `ebnf2tikz.pdf` (package documentation) in the project root. Additional targets:

```
make check          # Run AST-dump regression tests
make check-tikz     # Build PDF with all railroad diagrams
make check-sty      # Run LaTeX package integration tests
make sty            # Extract ebnf2tikz.sty from .dtx
make sty-doc        # Build ebnf2tikz.pdf package documentation
make doc            # Generate Doxygen API documentation
make realclean      # Remove all build artifacts
```

3 Quick Start

1. Create a LaTeX document that loads the package and includes an inline grammar:

```
\documentclass{article}
\usepackage{ebnf2tikz}
\begin{document}

\begin{ebnf}
greeting = 'hello' , [ name ] , '!' ;
name = identifier ;
\end{ebnf}
```

```
The greeting syntax is:
\useproduction{greeting}

\end{document}
```

2. Compile three times with shell escape enabled:

```
pdflatex -shell-escape mydocument
pdflatex -shell-escape mydocument
pdflatex -shell-escape mydocument
```

This multi-pass workflow is similar to the feedback loop required by BibTEX or `makeindex`. On the first pass, the `ebnf` environment writes the grammar to a file and `ebnf2tikz` runs with estimated node dimensions. LATEX then measures the actual rendered sizes of each node and writes them to `bnfnodes.dat`. On subsequent passes, `ebnf2tikz` reads these measurements and produces an accurate layout. See Section 7 for details.

4 Command-Line Options

Short	Long	Description
`-O`	`--optimize`	Enable optimization (default).
`-n`	`--nooptimize`	Disable optimization and subsumption.
`-f`	`--makefigures`	Wrap `tikzpicture`s in LATEX `figure` environments.
`-d`	`--dumponly`	Parse and dump the AST; do not generate TikZ output.
`-B`	`--dump-before`	Dump the AST before optimization (to stdout).
`-A`	`--dump-after`	Dump the AST after optimization (to stdout).
`-h`	`--help`	Print usage information and exit.

Usage:

```
./ebnf2tikz [options] input_file output_file
```

The `-O` flag is the default and is accepted for compatibility. Use `-n` to suppress both the optimizer and annotation-driven subsumption.

The `-B` and `-A` flags print the AST to standard output before and after optimization, respectively. These are useful for debugging grammars. The `-d` flag prints the AST and skips TikZ generation entirely.

5 EBNF Syntax Reference

`ebnf2tikz` accepts standard EBNF as defined by ISO 14977, with a few extensions.

5.1 Productions

A grammar consists of one or more productions. Each production has a name, an equals sign, a body expression, and a terminating semicolon:

```
production_name = body_expression ;
```

5.2 Constructs

Syntax	Meaning	Example
`A , B`	Sequence (concatenation)	`'if' , expr , 'then'`
`A | B`	Choice (alternation)	`'true' | 'false'`
`[ A ]`	Optional	`[ 'else' , statement ]`
`{ A }`	Repetition (zero or more)	`{ ',' , param }`
`( A )`	Grouping	`( 'a' | 'b' ) , 'c'`
`'text'`	Terminal (single quotes)	`'while'`
`"text"`	Terminal (double quotes)	`"begin"`
`? text ?`	Special sequence	`? any character ?`
`name`	Nonterminal reference	`expression`
\\	Explicit line break	Forces a new row in the diagram

5.3 Comments

Three comment styles are supported:

```
// C++-style line comment
--- Ada-style line comment
(* Block comment, can span
   multiple lines *)
```

6 Annotations

Annotations are placed immediately before a production definition using the `<<-- ... -->>` syntax. Multiple annotations may be placed before a single production.

6.1 `subsume`

```
«- subsume -»
production_name = body ;
```

The `subsume` annotation marks a production for inlining: wherever its name appears as a nonterminal reference in other productions, the reference is replaced by the production's body. The subsumed production itself is not emitted as a separate diagram.

6.2 `subsume as`

```
«- subsume as regex_pattern -»
production_name = body ;
```

Like `subsume`, but the production is only inlined into nonterminals whose names match the given POSIX extended regular expression.

6.3 `caption`

```
«-  caption "A descriptive caption"  -»
production_name = body ;
```

Sets the LaTeX figure caption for this production's diagram. Requires the `-f` flag to have any effect, since captions only apply to `figure` environments.

6.4 `label`

```
«-  label "fig:my-production"  -»
production_name = body ;
```

Sets the LaTeX `\label` for this production's figure, allowing cross-references with `\ref{fig:my-production}`. Requires the `-f` flag. The `\label` is always emitted after `\caption` in the generated output. Multiple annotations can be combined with semicolons:

```
«-  caption "The greeting syntax" ; label "fig:greeting"  -»
greeting = 'hello' , [ name ] , '!' ;
```

6.5 `sideways`

```
«-  sideways  -»
production_name = body ;
```

Places this production's diagram in a `sidewaysfigure` environment (from the `rotating` package) instead of a regular `figure`. Requires the `-f` flag. Can be combined with `caption` and `label`:

```
«-  sideways ; caption "Full SQL syntax" ; label "fig:sql"  -»
select_stmt = 'SELECT' , column_list , 'FROM' , table_ref ;
```

7 LaTeX Integration

The `ebnf2tikz` LaTeX package provides all required TikZ styles, commands, lengths, and the multi-pass infrastructure. Install `ebnf2tikz.sty` where LaTeX can find it (e.g., in your document directory or your local `texmf` tree), then add:

```
\usepackage{ebnf2tikz}
```

7.1 Package Options

Option	Effect
`figures`	Pass `-f` to `ebnf2tikz`, wrapping each diagram in a `figure` environment.
`nooptimize`	Pass `-n` to `ebnf2tikz`, disabling optimization and subsumption.

7.2 Providing EBNF Input

There are two ways to provide EBNF grammars to the package.

External files. Register one or more `.ebnf` files in the preamble or body:

```
\ebnffile{mygrammar.ebnf}
```

Inline grammars. Use the `ebnf` environment to write grammars directly in your LaTeX source:

```
\begin{ebnf}
greeting = 'hello' , [ name ] , '!' ;
name = identifier ;
\end{ebnf}
```

Content inside the `ebnf` environment is captured verbatim and written to `\jobname-rail.ebnf`.

7.3 Placing Diagrams

Use `\useproduction{name}` to place a specific production's railroad diagram:

```
The greeting syntax is:
\useproduction{greeting}
```

The name must match a production name from your EBNF input. When using `-f` (or the `figures` package option), diagrams are automatically wrapped in `figure` environments and do not require `\useproduction`.

7.4 Multi-Pass Workflow

With `-shell-escape` enabled, the package runs `ebnf2tikz` automatically at `\begin{document}`. The typical workflow is:

```
pdflatex -shell-escape mydocument    # captures EBNF, runs ebnf2tikz
pdflatex -shell-escape mydocument    # uses real node sizes from pass 1
pdflatex -shell-escape mydocument    # final accurate layout
```

Without `-shell-escape`, run `ebnf2tikz` manually between passes:

```
pdflatex mydocument
ebnf2tikz mydocument-rail.ebnf mydocument-rail.tex
```

```
pdflatex mydocument
ebnf2tikz mydocument-rail.ebnf mydocument-rail.tex
pdflatex mydocument
```

7.5 Customizable Lengths

The package defines several lengths with sensible defaults. Override them with `\setlength` after loading the package:

Length	Default	Purpose
`\railcolsep`	5pt	Horizontal gap between nodes.
`\railrowsep`	`\baselineskip`	Vertical gap between rows in multi-row diagrams.
`\railnodeheight`	14pt	Minimum height of terminal and nonterminal nodes.
`\railcorners`	4pt	Rounding radius for rail corners.

7.6 Customizable Styles

The package defines three TikZ styles that control the visual appearance of railroad diagrams.

The `raildiagram` style. Every railroad diagram is wrapped in `\begin{tikzpicture}[raildiagram]`. The `raildiagram` style sets the default line weight to `thick` for all rail connections, and establishes a baseline node style with zero padding and no visible border:

```
raildiagram/.style={
  thick,
  every node/.style={inner sep=0, outer sep=0, draw=none}
}
```

This style is *scoped* to railroad diagram pictures only. It does not affect any other `tikzpicture` environments in your document. You can freely mix railroad diagrams with circuit diagrams, plots, commutative diagrams, or any other TikZ content without style interference.

The `terminal` style. Draws a rounded rectangle with a subtle grey gradient fill. Applied to terminal (literal) tokens such as `'while'` or `"begin"`.

The `nonterminal` style. Draws a sharp-cornered rectangle with a red-tinted gradient fill. Applied to nonterminal references such as `expression` or `statement`.

Font commands. Three commands control the fonts used for text within diagrams:

Command	Default	Purpose
`\railtermname`	`\bfseries`	Terminal node text
`\railname`	`\itshape`	Nonterminal node text
`\railprodname`	`\itshape`	Production name labels

Use the setter commands `\setrailtermfont`, `\setrailnontermfont`, and `\setraillabelfont` to change these after loading the package.

8 Auto-Wrap

Productions whose body exceeds the available `\textwidth` are automatically broken across multiple rows in the railroad diagram. This happens without any user intervention.

Additionally, wide nonterminal labels that contain spaces are automatically wrapped using `\shortstack` to reduce their rendered width.

If you need to force a line break at a specific point, use the explicit line break operator `\\` in your EBNF input:

```
long_production =
    first_part , second_part , \\ third_part , fourth_part ;
```

9 Examples

The examples in this section are live—they are rendered by `ebnf2tikz` and LaTeX as part of building this manual.

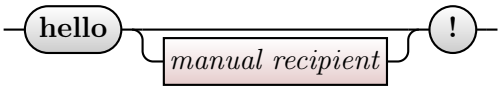
9.1 Simple Productions

A basic grammar with a sequence, a choice, and an optional element:

```
// Simple productions
manual_greeting = 'hello' , [ manual_recipient ] , '!' ;
manual_recipient = 'world' | manual_name ;
manual_name = first_name , [ last_name ] ;
```

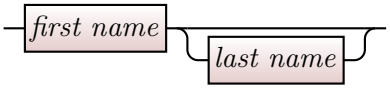
Rendered output:

manual greeting



manual recipient

manual name



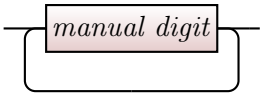
9.2 Repetition (Loops)

Repetition is expressed with curly braces. The optimizer transforms loops with separators into a compact form:

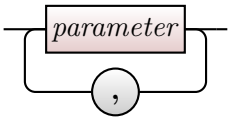
```
// Repetition
manual_number = manual_digit , { manual_digit } ;
manual_parameter_list = parameter , { ',' , parameter } ;
manual_digit = '0'|'1'|'2'|'3'|'4'|'5'|'6'|'7'|'8'|'9' ;
```

Rendered output:

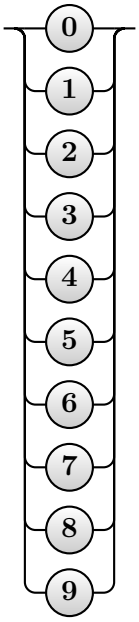
manual number



manual parameter list



manual digit



9.3 Subsumption

The `subsume` annotation inlines a production wherever it is referenced, then the optimizer simplifies the result. Consider this VHDL fragment:

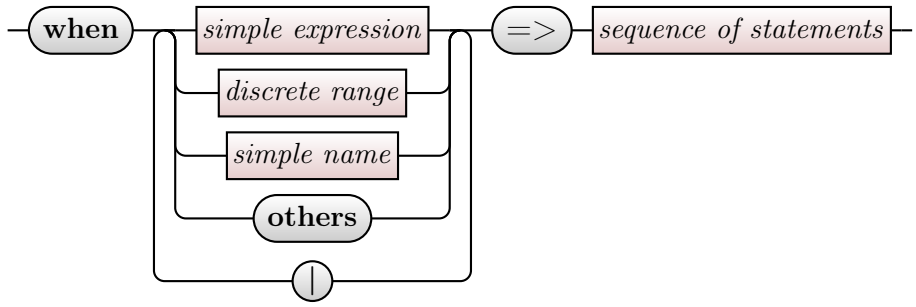
```
case_statement_alternative =
    'when' , choices , '=>', sequence_of_statements;

 «-  subsume  -»
choices = choice, { '|', choice } ;

 «-  subsume  -»
choice = simple_expression | discrete_range
       | simple_name | 'others' ;
```

After subsumption and optimization, only a single compact diagram is produced for `manual_case_alt`:

manual case alt



Without subsumption, the diagram would instead show `choices` and `choice` as opaque nonterminal references, requiring the reader to look up separate diagrams.

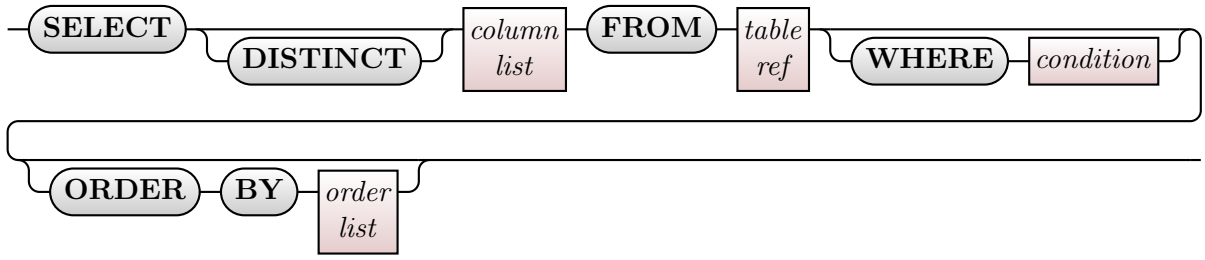
9.4 Complex Example with Nested Constructs

This example combines sequences, choices, optionals, and loops:

```
select_stmt = 'SELECT' , [ 'DISTINCT' ] ,
    column_list , 'FROM' , table_ref ,
    [ 'WHERE' , condition ] ,
    [ 'ORDER' , 'BY' , order_list ] ;
```

Rendered output:

manual select stmt



10 Customization Examples

All railroad diagrams are generated inside `\begin{tikzpicture}[raildiagram]`, so the `raildiagram` TikZ style controls the overall appearance of every diagram. Because it is applied as an option to individual `tikzpicture` environments, it is completely scoped—changes to `raildiagram` affect only railroad diagrams, never other TikZ pictures in the document.

10.1 Default Appearance

With no customization, the default `raildiagram` style uses `thick` lines, terminals are grey rounded rectangles, and nonterminals are pink-tinted rectangles:

```
% No customization needed --- this is the default
\usepackage{ebnf2tikz}
```

All of the diagrams in Section 9 use the default appearance.

10.2 Changing Line Weight

To make railroad diagrams use thinner lines, append to the `raildiagram` style:

```
\tikzset{raildiagram/.append style={thin}}
```

To change the line weight for individual node types as well, also update the `terminal` and `nonterminal` styles:

```
\tikzset{
  raildiagram/.append style={thin},
  terminal/.append style={thin},
  nonterminal/.append style={thin}
}
```

10.3 Changing Node Colors

Override the gradient fills on terminal and nonterminal nodes:

```
\tikzset{
  terminal/.append style={
    top color=white, bottom color=blue!20
  },
  nonterminal/.append style={
    top color=white, bottom color=green!20
  }
}
```

For flat (non-gradient) fills, replace the shading keys entirely:

```
\tikzset{
  terminal/.style={
    draw, inner sep=2pt, rounded rectangle,
    minimum size=\railnodeheight, thick,
    fill=orange!15
  },
  nonterminal/.style={
    draw, inner sep=2pt, rectangle,
    minimum size=\railnodeheight, thick,
    fill=cyan!15
  }
}
```

10.4 Changing Fonts

Use the setter commands to change the font used for text inside nodes and production labels:

```
\setrailtermfont{\ttfamily\bfseries}      % monospace bold terminals
```

```
\setrailnontermfont{\sffamily\itshape}   % sans-serif italic
    nonterminals
\setraillabelfont{\sffamily\bfseries}    % sans-serif bold labels
```

10.5 Changing Spacing

Adjust the spacing parameters after loading the package:

```
\setlength{\railcolsep}{8pt}            % wider horizontal gaps
\setlength{\railrowsep}{20pt}           % taller vertical gaps
\setlength{\railnodeheight}{18pt}       % taller nodes
\setlength{\railcorners}{6pt}           % rounder corners
```

Note: after changing these lengths, you must re-run the full `ebnf2tikz` → `pdflatex` feedback loop, since the layout engine reads spacing parameters from the node sizes file.

10.6 Replacing the raildiagram Style Entirely

If you need full control, you can completely replace the `raildiagram` style. When doing so, you *must* include the baseline `every node` reset, because the generated TikZ code depends on nodes having zero default padding:

```
\tikzset{
  raildiagram/.style={
    semithick,
    every node/.style={inner sep=0, outer sep=0, draw=none}
  }
}
```

Omitting the `every node` reset will cause unwanted padding and borders on the coordinate nodes and connection points that form the railroad tracks.

10.7 Coexistence with Other TikZ Packages

Because `raildiagram` is scoped to railroad pictures, it does not interfere with other TikZ-based packages. For example, you can use `ebnf2tikz` alongside `circuitikz`, `pgfplots`, or `tikz-cd` in the same document without any style conflicts:

```
\documentclass{article}
\usepackage{ebnf2tikz}
\usepackage{circuitikz}     % uses its own TikZ styles
\usepackage{pgfplots}       % uses its own TikZ styles
\ebnffile{mygrammar.ebnf}

\begin{document}

Railroad diagram (uses raildiagram style):
\useproduction{my_rule}

Circuit diagram (unaffected by ebnf2tikz):
\begin{circuitikz}
```

```
  \draw (0,0) to[R, l=$R_1$] (2,0) to[C, l=$C_1$] (2,-2) -- (0,-2);
\end{circuitikz}

Plot (unaffected by ebnf2tikz):
\begin{tikzpicture}
  \begin{axis}[width=6cm, height=4cm]
    \addplot {x^2};
  \end{axis}
\end{tikzpicture}

\end{document}
```

11 Tips

- Use the `-B` and `-A` flags to inspect the AST before and after optimization. This is helpful for understanding how the optimizer transforms your grammar.
- When adding `ebnf2tikz` diagrams to an existing document, it is convenient to `\input` the generated TikZ file rather than pasting it inline.
- The `-f` flag wraps each diagram in a `figure` environment, which enables captions, labels, and `\listoffigures`. Without `-f`, diagrams are placed inline.
- Production names must be valid identifiers (letters, digits, underscores). Underscores in production names are rendered in the diagram using LaTeX formatting.
- The feedback loop (`ebnf2tikz` → `pdflatex` → `ebnf2tikz` → `pdflatex`) need only be repeated when node content changes. If you are only editing prose around existing diagrams, a single `pdflatex` run suffices.